CONFIDENTIAL IMPLEMENTATION SPECIFICATION · PREPARED 25 JULY 2026

OpenClaw Handoff Control Plane v2

A complete, build-ready specification for turning the shipped handoff plugin into a singular, transactional, event-driven orchestration control plane with safe autonomous delivery, exact evidence, fast baton passes, bounded resource use, and deterministic recovery.

> **Build decision:** preserve the proven handoff domain model and the Phase 0 safety foundation in version `0.1.100`. Extract a small typed kernel behind compatibility adapters. Do not add another controller, sweeper, model-callable shell path, or independent state mirror.

IMPLEMENTATION TARGET **v2.0.0 control plane**	REQUIRED SOURCE BASELINE **0.1.100 / schema 2026-07-25.1**
PRIMARY OPERATOR **Oscar**	FIRST PROVING CUSTOMER **Elliott Wave Trader**

This document authorizes no implementation, install, restart, provider write, card creation, or cleanup by itself. It is intended to be given to a high-reasoning Codex implementation session.

DOCUMENT CONTROL

How Codex should use this specification

Treat this as the product and architecture contract for v2. The ordered work packages, invariants, durable schemas, tests, and exit gates are normative. File names are recommended seams, not permission to overwrite a dirty checkout.

> **Critical baseline warning.** The inspected local primary checkout was still on a dirty `0.1.99` branch, while the shipped Phase 0 implementation is `0.1.100` from PR #54. A stale `origin/main` reference also reported `0.1.96`. Codex must fetch, locate, and verify the merged PR #54 or an exact `0.1.100` release ref before creating its implementation worktree. It must not build v2 from the older dirty checkout or assume `origin/main` is current.

Mandatory preflight

1. Run the repository's status, branch, worktree, tag/ref, and version audit without changing files.
2. Resolve the exact commit containing package `0.1.100`, schema `2026-07-25.1`, Phase 0 execution context, fixed deployment operations, durable effect intents, and ToolSpec parity.
3. Create a dedicated v2 worktree and branch from that exact commit. Preserve every existing dirty checkout, worktree, branch, untracked file, installed runtime, and state directory.
4. Run the complete baseline proof suite before the first source edit and record the exact test count.
5. Create one source-of-truth GitHub epic with child issues matching the work packages in this document.
6. Do not install or restart the live plugin until the final canary gate. Use the mandatory safe gateway restart procedure; one rollout receives at most one gateway restart.

Normative language

Term	Meaning
MUST / MUST NOT	Required for v2 completion. Failure blocks merge or rollout.
SHOULD	Expected design. A deviation requires an architecture decision record and equivalent proof.
MAY	Optional implementation detail that cannot weaken an invariant.
Provider-authoritative	Confirmed by the external system's receipt, cursor, delivery ID, status API, or immutable artifact—not by model narration.

144
current optional public tools to collapse behind a smaller kernel

50,189
current `src/tools.ts` lines on the Phase 0 branch

< 2 min
target result-to-next-node dispatch p95

REPORT MAP

Contents

Recommended reading order: read Sections 01–05 before coding; implement Sections 06–18 in the phase order in Section 24; use Sections 21–25 as hard release gates; paste Section 27 into the implementation session together with this PDF.

SECTION 01

Executive completion verdict

Version 0.1.100 finished the safety foundation. Version 2 is complete only when the plugin becomes the single durable mutation authority for a canonical story family and can move a typed result to the next eligible assignment quickly, idempotently, and recoverably.

What v2 must deliver

Singular control

Exactly one leased controller owns mutations for a canonical issue family. Hooks, cron, Workboard, Slack, GitHub, canaries, and safety sweeps can enqueue facts; none can independently advance story state.

Transactional truth

Active orchestration state lives in a transactional journal with uniqueness, fencing, append-only evidence, crash-safe effect ordering, integrity verification, and deterministic replay.

Immediate baton pass

A valid result commits node completion and newly ready events atomically. The next eligible node is admitted without waiting for a five-minute broad sweep.

Bounded autonomy

Workers receive short-lived capabilities for one node, exact run, exact paths, allowed commands, risk ceiling, resource limits, and expiry. Policy—not a model—decides promotion.

Two separate definitions of done

Boundary	Done when	Not required
Handoff plugin v2	Kernel, database, inbox, controller, DAG, capabilities, effects, evidence, hooks, compatibility, operator controls, recovery, and production canary all meet the gates in this document.	A promoted EWT ML model or completion of EWT product development.
EWT first vertical slice	One bounded EWT story is ingested, compiled, implemented, independently proved, advanced, closed, and recovered through v2 with exact artifacts.	Claiming that the plugin owns Elliott Wave truth or that ML may override deterministic invalidation.

Bottom line: v2 is an extraction and hardening program, not a greenfield workflow rewrite. Keep the canonical identity, typed lifecycle, proof profiles, run/generation fences, claim-before-I/O, durable receipts, and chaos mindset already present.

SECTION 02

Shipped baseline and preservation boundary

Codex should not spend v2 time rebuilding Phase 0. It should prove those properties remain intact while moving them behind the new transactional kernel.

Already shipped in 0.1.100

- ✓ Model-callable free-form rollout shell strings were replaced by fixed deployment operations.
- ✓ `ExecutionContext` carries deadline, abort signal, trace identity, and trusted principal.
- ✓ Privileged mutations require server-derived authority and scoped principals.
- ✓ External effect intents are persisted before I/O; ambiguous outcomes are retained as unknown.
- ✓ Exact reconciliation evidence binds the effect key and payload hash.
- ✓ Unknown dispatch does not falsely report a healthy admitted run.
- ✓ Runtime, manifest, Gateway, schema, and documentation parity are checked against a ToolSpec registry.
- ✓ Safe rollout uses the guarded restart procedure and passed live verification.
- ✓ Phase 0 passed 46 test files / 768 tests, hosted CI, independent review, and live parity.

Current structural debt to remove

Observed seam	Why it remains a v2 blocker	Required treatment
`src/tools.ts` ≈ 50,189 lines	Policy, schemas, composition, adapters, and orchestration are entangled.	Extract commands, queries, policies, projectors, and compatibility adapters by bounded slices.
`src/store.ts` ≈ 5,335 lines	Whole-envelope JSON rewrites, process-local queues, and active/archive repair are expensive and hard to transact across processes.	Introduce a single-writer SQLite kernel with WAL, migrations, unique constraints, and append-only journals.
144 optional tools	High schema load, overlapping meanings, recursive composition, drift risk, and weak operator discoverability.	Expose 8–12 public v2 tools; retain versioned compatibility aliases for two releases.
Multiple controllers and initiators	Story controller, hub controller, ML controller, hooks, sweeps, cron, and canaries can compete.	Make one family saga controller the only mutation authority; every other initiator enqueues.
JSON effect intents	Phase 0 semantics are correct but persistence is still outside a transactional story/effect commit.	Move effects into the same database transaction boundary as controller decisions.
Dependency-key closeout defect (#55)	Historical stale dependency identifiers can block bookkeeping transitions.	Repair before cutover; add canonical key mapping and migration fixtures.

Preservation rule

> No “simplification” may replace typed evidence with Slack prose, caller-selected identity, prompt-based completion, best-effort state mirrors, blind retries, mutable historical evidence, or process-local locks. Compatibility adapters may translate old calls; they may not recreate a second truth path.

SECTION 03

Definition of v2 complete

The following twelve capabilities are jointly required. Partial delivery may be useful, but it is not v2 complete.

#	Capability	Completion evidence
1	Transactional active state	SQLite/WAL authoritative; integrity, backup, restore, and restart convergence proven.
2	Durable event inbox	Verified, deduplicated, cursor/revision-bound provider events persist before action.
3	Single family controller	Unique lease/fencing token prevents split brain and stale writes.
4	Versioned DAG compiler	Typed cards compile deterministically; migrations append graph versions.
5	Bounded dispatch	WIP, conflict, resource, deadline, run, generation, and cancellation enforced.
6	Unified identity/capabilities	One registry binds agent, Slack, session, model, role, health, risk, and receiving state.
7	Transactional effects	Intent-before-I/O, unknown reconciliation, receipts, stable keys, and no blind replay for every provider.
8	Immutable evidence	Artifacts, SHAs, CI, QA, model route, proof verdict, and lineage are exact and append-only.
9	Small public kernel	8–12 tools cover v2 control; legacy tools are generated compatibility wrappers with telemetry.
10	Event-driven advancement	Result-to-next p95 < 2 minutes; broad sweep causes < 10% of normal transitions.
11	Operator clarity	Oscar sees one canonical queue, reason codes, effect ambiguity, WIP, health, deadlines, decisions, and proof gaps.
12	Safe production canary	Low-risk story closes end-to-end; crash/restart replay emits no duplicate work or provider effect.

Service targets

- Hook enqueue p99: **< 50 ms**
- Result-to-ready event p99: **< 5 s**
- Result-to-next dispatch p95: **< 2 min**
- Controller decision p95: **< 500 ms** excluding provider I/O
- Graceful shutdown: **< 10 s**

Correctness targets

- Duplicate node launches: **0**
- Stale result transitions: **0**
- Blind retries of unknown effects: **0**
- High-risk downgraded auto-completions: **0**
- Second recovery-pass repairs: **0**

SECTION 04

Executable control-plane invariants

Each invariant must exist as a database constraint, typed policy, or deterministic test—not only as documentation.

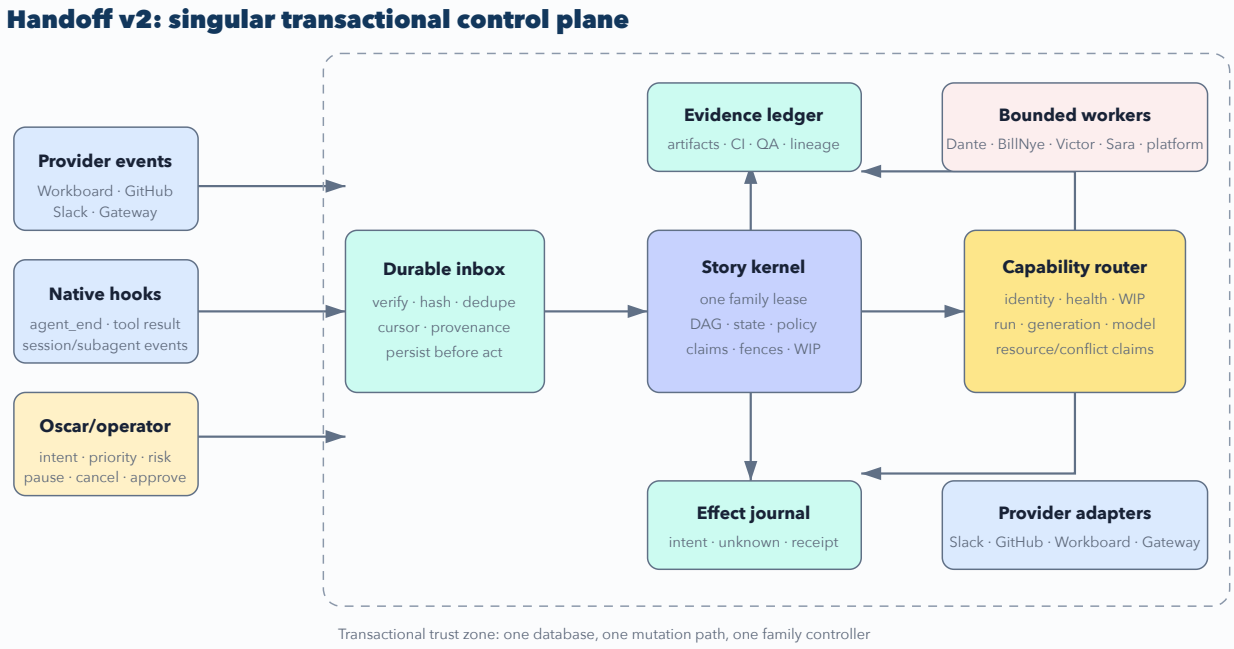
ID	Invariant	Primary enforcement	Required failure test
I-01	One canonical story per issue family.	Unique canonical family key and duplicate-card link.	Concurrent intake of two equivalent cards yields one story.
I-02	One active controller lease per family.	Lease generation plus fenced compare-and-swap.	Old controller cannot commit after lease takeover.
I-03	Persist before act.	Event/effect transaction commits before provider call.	Crash between commit and I/O recovers without data loss.
I-04	Claim before I/O.	Node claim, assignment, run, generation, and deadline commit atomically.	Two schedulers cannot launch the same node.
I-05	Exact evidence binding.	Foreign keys to story, graph, node, assignment, run, generation, artifact hash, proof profile.	Cross-story or cross-generation proof is rejected.
I-06	Stale work is harmless.	Fencing token and generation checks on every mutation.	Late successful worker cannot advance superseded work.
I-07	Ambiguity is retained.	Unknown effect status with reconciliation policy and next attempt.	Timeout after provider acceptance produces no duplicate retry.
I-08	Models propose; policy decides.	Deterministic admission, completion, merge, promotion, and risk gates.	High model confidence cannot bypass missing proof.
I-09	One active worktree/conflict owner.	Unique active resource claim by normalized key.	Conflicting assignments serialize.
I-10	Every mutation is idempotent.	Command key plus payload hash and original receipt.	Retry returns identical receipt; collision fails closed.
I-11	Hooks only enqueue.	No provider, model, controller, or file-store mutation inside hook path.	Hook stays bounded while providers are unavailable.
I-12	Restart converges.	Journal replay, lease expiry, effect reconciliation, deterministic projectors.	Second reconciliation pass makes zero repairs.
I-13	History is append-only.	Events/evidence cannot be updated or deleted by normal commands.	Attempted evidence rewrite is rejected and audited.
I-14	One command path mutates state.	Adapters call kernel commands; legacy tools cannot write stores directly.	Static/dependency test blocks direct adapter writes.
I-15	Authority is server-derived.	Authenticated principal and delegated capability intersection.	Caller-supplied actor, owner, or webhook fields cannot elevate scope.
I-16	Deletion is policy-bound and recoverable.	Retention tombstone, export checkpoint, reference check, and audit receipt.	Referenced records cannot be compacted.

> **Design review question:** every new v2 feature must name its owning invariant, durable records, idempotency key, authorization scope, timeout, failure state, retry/reconciliation policy, observability fields, and restart behavior.

SECTION 05

Target architecture

Sources publish facts. One transactional kernel decides. Providers receive journaled effects. Workers mutate only their assignment.



Connectors are drawn before nodes and all orchestration truth remains inside the transactional trust zone.

Layer responsibilities

Layer	Responsible for	Explicitly forbidden
Adapters	Authentication, validation, normalization, provider I/O, receipt parsing.	Deciding story readiness, completion, retry, owner, or promotion.
Inbox/projectors	Immutable receipt, dedupe, cursor, normalized facts, read models.	Launching workers or sending provider effects.
Kernel	Commands, policy, state transitions, claims, graph readiness, proof verdicts.	Direct network I/O or model calls inside a database transaction.
Controller	Serial event evaluation for one family and effect/ready-event creation.	Mutating without a live fenced lease.
Router	Capability intersection, health/WIP/resource selection, route receipt.	Weakening proof because a preferred model or owner is unavailable.

SECTION 06

Typed command/query kernel

The kernel is the only code allowed to mutate control-plane state. Public tools, Gateway methods, hooks, cron, Workboard, Slack, and compatibility aliases translate requests into typed commands or queries.

Recommended public v2 surface

Tool	Command/query	Purpose	Authority
`card.submit`	Command	Validate a typed card, canonicalize family, compile first graph version, return receipt.	intent.write
`event.ingest`	Command	Accept verified provider/native event through a trusted adapter.	adapter.ingest
`story.get`	Query	Canonical story, graph, nodes, evidence, blockers, effects, why-trace.	operator.read
`story.list`	Query	Queue with cursor, owner, WIP, readiness, risk, health, next trigger.	operator.read
`story.control`	Command	Pause, resume, cancel, supersede, retry, reprioritize, approve exception.	story.control + risk
`result.submit`	Command	ACK, progress, blocker, or terminal result for exact run/generation.	assignment.result
`evidence.submit`	Command	Attach artifact/CI/QA/provider/model receipts to exact proof profile.	assignment.evidence
`operator.snapshot`	Query	Compact control frame, queue age, baton latency, conflicts, decisions, anomalies.	operator.read
`runtime.verify`	Query	Database, migration, catalog, schema, hook, provider, backup, recovery health.	operator.read
`deployment.apply`	Command	Fixed supervisor operation with drain, rollback, health, and one-restart gate.	deployment.apply

Internal command envelope

```
{
  command_id: "cmd_...",
  idempotency_key: "provider:event:revision | operator:request",
  command_type: "story.node.claim.v2",
  schema_version: 2,
  principal_id: "server-derived",
  delegation_id: "optional exact capability",
  trace_id: "trace_...",
  story_id: "story_...",
  expected_story_version: 42,
  controller_fence: 7,
  payload_hash: "sha256:...",
  occurred_at: "...",
  received_at: "...",
  deadline_at: "...",
  payload: { ... }
}
```

Kernel rules

- Every command validates schema, principal, capability, expected version, fence, idempotency key, and payload hash.
- The transaction writes the command receipt, domain events, state changes, ready events, and effect intents atomically.
- Provider/model I/O occurs after commit. Its result returns through a new event/command.
- Queries read versioned projections and never trigger maintenance or mutations as a side effect.
- Old tools call these commands/queries in-process; they never call another registered model tool.
- Static architecture tests reject imports from adapters into kernel policy and reject store writes outside repositories.

SECTION 07

Transactional SQLite persistence

Use SQLite WAL as the authoritative active control plane. The current runtime is Node 24, so the preferred first choice is the built-in `node:sqlite` driver to avoid a native dependency; gate startup on the minimum supported Node runtime and prove backup/restore behavior on the Mac mini.

Database operating contract

- Single database file inside the plugin state directory; WAL enabled; foreign keys enabled; synchronous mode appropriate for control-plane durability.
- One logical writer service. Concurrent callers submit kernel commands; no adapter opens an independent write path.
- Explicit transactions; bounded busy timeout; prepared statements; schema migrations under exclusive migration lease.
- Stable UTC timestamps plus monotonic sequence numbers for ordering. Wall clock alone never decides event order.
- Startup performs schema/version check, quick integrity check, migration checksum verification, and last-shutdown/recovery inspection.
- Online backup uses SQLite's consistent backup mechanism or a proven checkpoint/copy procedure; never copy a live database and WAL naively.
- Disk-full, corruption, busy-timeout, and read-only failures fail closed with actionable health state; no fallback to uncoordinated JSON writes.

Core tables

Table	Essential keys	Purpose
`schema_migrations`	version PK, checksum, applied_at, build	Immutable migration history.
`command_receipts`	command_id PK, idempotency_key UNIQUE, payload_hash	Idempotent mutation entry and original result.
`event_inbox`	source + event_id + revision UNIQUE	Verified persist-before-act provider/native events.
`stories`	story_id PK, canonical_family_key UNIQUE, version	Canonical intent, priority, risk, status, controller generation.
`graph_versions`	story_id + graph_version UNIQUE	Immutable compiled DAG versions and compiler receipt.
`nodes`	story_id + graph_version + node_id UNIQUE	Materialized node state and proof/risk/resource contracts.
`node_dependencies`	node PK/FK + dependency PK/FK	Typed edges, join policy, optionality.
`assignments`	assignment_id PK, run_id UNIQUE, node FK	Owner, generation, deadline, capability, exact output contract.
`leases`	scope_key PK, generation, expires_at	Controller, owner, worktree, resource, and migration fences.
`ready_events`	ready_id PK, node/generation UNIQUE	Transactional outbox for event-driven admission.
`effect_intents`	idempotency_key PK, payload_hash	Pending/confirmed/unknown/failed external effects.
`effect_attempts`	effect_key + attempt_no UNIQUE	Attempt timing, transport result, reconciliation evidence.

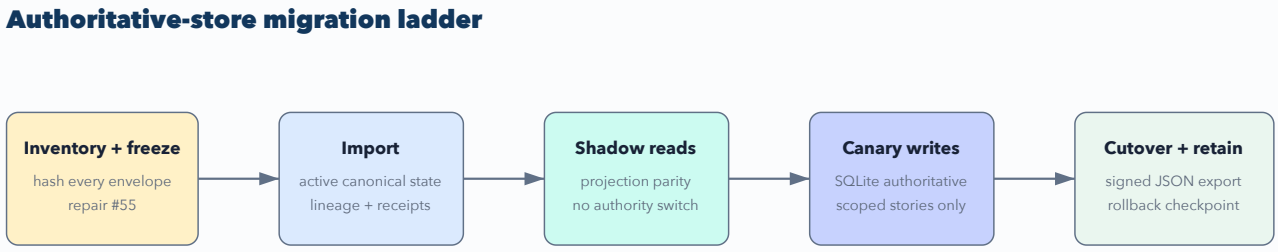
`evidence_events`	evidence_id PK, content_hash, exact run/gen FKs	Append-only artifacts, CI, QA, proof and lineage.
`dead_letters`	dead_letter_id PK, source reference	Original input, reason, attempts, next policy, operator state.
`agent_capabilities`	agent_id + manifest_version UNIQUE	Versioned authoritative owner registry.
`delegations`	delegation_id PK, nonce UNIQUE	Exact bounded authority chain.
`metrics_events`	metric_id PK, sequence	Append-only latency, WIP, health, resource, and outcome measurements.
`audit_events`	audit_id PK, sequence	Security, policy, exception, migration, and operator mutation trace.

> Do not reproduce twenty-two JSON envelopes as twenty-two unrelated SQL tables. Design around domain invariants and transactions. Historical JSON remains immutable migration evidence, not a second active store.

SECTION 08

JSON-to-SQLite migration

Migration must be reversible, parity-measured, and intentionally selective. Import canonical active state and required history; retain legacy envelopes as signed read-only evidence.



Required migration phases

1. **Inventory:** enumerate every active, archive, DLQ, action, receipt, effect, lease, binding, and backup envelope; record size, record count, schema, hash, duplicates, dangling references, and oldest/newest timestamps.
2. **Repair gate:** fix issue #55 and every dangling canonical dependency key. Produce deterministic fixtures for the historical failure.
3. **Export checkpoint:** pause mutations briefly; take a signed manifest of source files, hashes, build, schema, timestamps, counts, and backup location. Resume only after verification.
4. **Import:** load canonical active stories, required assignment/evidence/effect history, unresolved unknowns, live dependencies, leases, and operator decisions. Do not import completed Workboard noise as active truth.
5. **Parity:** run existing queries through JSON and SQLite repositories and compare normalized results. Classify every difference as expected transformation or blocker.
6. **Shadow mode:** keep JSON authoritative; exercise SQLite projections under live-like reads. No dual independent controllers.
7. **Scoped canary:** make SQLite authoritative for isolated low-risk stories while legacy reads remain comparison-only.
8. **Cutover:** one configuration switch makes SQLite authoritative. JSON becomes read-only. There is no indefinite dual-write mode.
9. **Rollback:** before irreversible schema advancement, prove rollback to the signed checkpoint and re-import of post-checkpoint events. Rollback must not duplicate provider effects.
10. **Retention:** preserve the original envelopes for the agreed audit window. Remove only through an explicit, recoverable operator-approved cleanup outside the v2 critical path.

Parity gate

Counts alone are insufficient. Compare canonical family election, visible lane, dependency readiness, assignment/run state, evidence completeness, effect status, unknown reconciliation, SLA/queue age, operator digest, archive protection, and exact why-traces.

SECTION 09

Durable event inbox

Every event that may affect orchestration must be persisted and deduplicated before a controller sees it.

Normalized event envelope

```
{
  source: "github | workboard | slack | gateway | hook | operator | timer",
  event_id: "provider delivery id or stable derived id",
  revision: "cursor/revision/version",
  schema_version: 2,
  event_type: "agent.result.submitted",
  payload_hash: "sha256:canonical-json",
  signature_status: "verified | trusted-runtime | unavailable",
  principal_id: "server-derived",
  occurred_at: "...",
  received_at: "...",
  source_cursor: "...",
  trace_id: "...",
  payload: { ... }
}
```

Source-specific rules

Source	Identity/dedupe	Verification	Failure policy
GitHub webhook	Delivery header + event + action/revision	Signature and repository allowlist	Persist then acknowledge; invalid signature rejected and audited.
Workboard	Board/card event ID or cursor + revision	Connected runtime principal and expected board	Cursor advances only after durable page import.
Slack	team/channel/message/thread/edit revision	Oscar account, allowed channel, sender mapping	Prose becomes an observation; only typed result syntax or explicit binding becomes a command.
Gateway/session	session/run/tool call/event sequence	Trusted runtime client identity	Unknown run identity cannot advance a node.
Native hook	runtime event ID + session/run sequence	In-process trusted source	Bounded enqueue only; failures go to local emergency queue/health alert.
Timer	policy + scope + scheduled instant	Controller-created durable timer	Duplicate wake is harmless and returns prior receipt.

Processing model

- Inbox records transition from `received` to `claimed` to `processed` or `dead_lettered`.
- A claim uses a lease/fence so restart can safely reclaim abandoned work.
- Processing creates kernel commands; it does not directly update story tables.
- Poison events retain original payload hash, parser version, error class, attempts, next policy, and operator disposition.
- Schema upgrades use versioned normalizers; old events remain readable and are never rewritten.

SECTION 10

Canonical story-family controller

One serial saga controller owns a canonical family. It reacts to durable events, evaluates deterministic policy, and atomically writes state transitions, newly ready events, effect intents, and evidence requirements.

Lease and fencing protocol

1. Controller attempts to acquire `story-family:<canonical-key>` with a monotonically increasing generation.
2. Every mutation supplies the lease generation and expected story version.
3. Renewal occurs before one-third of TTL remains; the lease is never treated as ownership after expiry.
4. Takeover increments generation. Writes from the former generation fail even if the former process resumes.
5. Controller releases voluntarily on terminal close, pause, clean shutdown, or supersession. Crash relies on expiry and recovery.

Event evaluation transaction

```
BEGIN IMMEDIATE;
  assert live lease + fence;
  load story/version + graph + reachable nodes + proof/effect summaries;
  validate incoming event and idempotency receipt;
  derive deterministic domain transition;
  append domain/audit events;
  update story/node/assignment projection with version increment;
  append newly-ready node events;
  append effect intents and durable timers;
  mark inbox event processed;
COMMIT;
-- only now may dispatcher/provider adapters perform I/O
```

Controller reason codes

Every no-op or transition returns a stable machine reason such as `DEPENDENCY_PROOF_MISSING`, `RESOURCE_CONFLICT`, `STALE_GENERATION`, `EFFECT_UNKNOWN`, `QA_INDEPENDENCE_REQUIRED`, `HUMAN_APPROVAL_REQUIRED`, `OWNER_UNHEALTHY`, or `RETRY_BUDGET_EXHAUSTED`.

What the controller must replace

- Independent story, hub, issue-family, ML, scheduler, canary, and sweep mutation logic.
- Free-form “next action” interpretation as a readiness mechanism.
- Controller-to-controller registered-tool calls.
- Broad scans as the normal advancement path.

> Keep domain-specific planning strategies—ML experiment planning, QA packet generation, closeout policy—as pure planners/policies invoked by the family controller. They must not retain independent leases or state-transition authority.

SECTION 11

Card compiler and versioned DAG

A typed card describes intent and done conditions. A deterministic compiler produces a versioned lazy DAG. The graph shape is known at intake, but assignments materialize only when dependencies, proof, authority, and resources are satisfied.

CardV2 required fields

Group	Fields
Identity	external source, card ID/revision, repository/project, issue family, canonicalization hints
Intent	outcome, user value, scope, non-goals, priority, risk, acceptance criteria
Dependencies	source artifacts, decisions, upstream stories, environmental prerequisites
Execution	capability queries, allowed repository/path scope, resource classes, deadlines, budgets
Proof	proof profile, independent reviewer, required CI/QA/provider receipts, closeout requirements
Human boundary	approval points, cost/access/privacy/production/architecture flags
Lifecycle	supersession key, cancellation semantics, report-back trigger, stop conditions

DAG node contract

```
{
  node_id: "implementation.api-contract",
  node_schema_version: 2,
  kind: "plan | implement | data | ml | qa | closeout | deploy",
  capability_query: ["typescript", "ewt_domain"],
  input_evidence: ["evidence:architecture-contract:v3"],
  output_contract: { schema: "...", paths: ["..."] },
  proof_profile: "implementation_v2",
  done_when: ["artifact_hash_bound", "ci_green", "independent_qa"],
  dependencies: [{ node: "architecture", join: "all_required" }],
  risk: "medium",
  approval_policy: "auto_after_independent_qa",
  resource_claims: ["repo:path:src/api", "worktree:issue-123"],
  budgets: { attempts: 3, wall_minutes: 45, external_cost_usd: 0 },
  stop_conditions: ["scope_drift", "dirty_owner_conflict", "rss_critical"]
}
```

Compiler requirements

- Same card revision + compiler version + policy pack produces the same graph hash.
- Ambiguous or incomplete cards enter typed triage; they do not produce partial runnable graphs.
- Cycles, missing outputs, incompatible proof profiles, unbounded fanout, and impossible capability queries fail at compile time.
- Graph changes append a new version and migration event. They never rewrite prior nodes or evidence.
- Supersession increments story and assignment generation, cancels eligible work, and fences late results.
- QA nodes are not materialized before a complete proof packet exists; expensive training is not admitted before corpus/data gates pass.

SECTION 12

Node admission, dispatch, and WIP

A ready dependency state is necessary but insufficient. Admission must also prove authority, owner health, conflict freedom, resource capacity, risk, and budget.

Admission order

1. Story active, graph current, node reachable, dependencies and join policy satisfied.
2. Required input evidence exists and matches exact graph/story versions.
3. Risk and approval policy permit autonomous admission.
4. Candidate owners have fresh receiving manifests and required capabilities.
5. Per-owner and global WIP limits permit work.
6. Worktree/path/dataset/deployment conflict claims are free.
7. CPU, RSS, disk, gateway, provider, and training budgets are healthy.
8. Controller atomically claims node and resources; creates assignment run/generation/delegation/deadline.
9. Dispatcher journals the exact Gateway/session effect and starts or resumes the worker.

Initial production limits

Limit	Initial value	Expansion gate
Global active workers	2–3	Soak proves gateway/event-loop/CPU/RSS/disk health and no duplicate launch.
Implementation WIP per owner	1	Evidence shows bounded parallel paths without branch conflicts.
Independent QA WIP	1	QA queue age and proof quality remain within SLO.
ML training	1 global	Resource isolation, cancellation, checkpoint, and cost reporting proven.
Deployment/restart	1 global exclusive	Never expanded.
Controller event batch	Bounded count/time	No starvation; event-loop p99 remains healthy.

Dispatch contract

The worker packet must contain the exact story/node/run/generation, capability/delegation ID, source ref, worktree/path scope, inputs, expected output schema/path, proof profile, deadline, progress cadence, blocker format, stop conditions, resource budget, and report-back endpoint. Slack prose is supplementary.

> If a session send times out or returns an ambiguous result, the assignment becomes `dispatch_unknown`. It cannot be launched again until the Gateway/session reconciler proves absence or binds the exact admitted run.

SECTION 13

Identity and delegation capabilities

One authoritative registry must bind OpenClaw agent identity, Slack account, session identity, role, capability, model policy, health, WIP, conflict groups, risk ceiling, and receiving status.

Capability manifest

Field	Requirement
Identity	agent ID, runtime owner ID, allowed Slack account, session namespace, repository identities
Capability	typed capability labels, allowed command classes, path scopes, provider destinations
Safety	risk ceiling, review independence classes, deployment prohibition, cost ceiling, privacy class
Runtime	receiving status, freshness/expiry, heartbeat, current WIP, conflict groups, resource class
Models	task classes, requested tiers, allowed fallback tiers, high-risk review constraints
Provenance	manifest version/hash, issuer, approved time, effective time, revocation status

Delegation capability

Use a tamper-evident server-issued capability record. A database-backed opaque token with a keyed MAC is acceptable; a full public-key system is not required unless tokens cross trust boundaries. The record must include:

- principal, agent, session namespace, parent delegation, story, graph, node, assignment, run, generation;
- allowed commands, repository/path prefixes, artifact destinations, Slack/provider recipients;
- risk and approval ceiling, model class ceiling, WIP/resource/cost/time budget;
- delegation depth, issued/expiry time, single-use or monotonic nonce, revocation state;
- controller fence and manifest version used at admission.

Authorization algorithm

```
effective_scope =
  authenticated_principal
  ∩ agent_manifest
  ∩ node_contract
  ∩ delegation_capability
  ∩ selected_model_risk_ceiling
  ∩ current_operator_policy;

deny if any required permission is absent;
never union permissions from caller input.
```

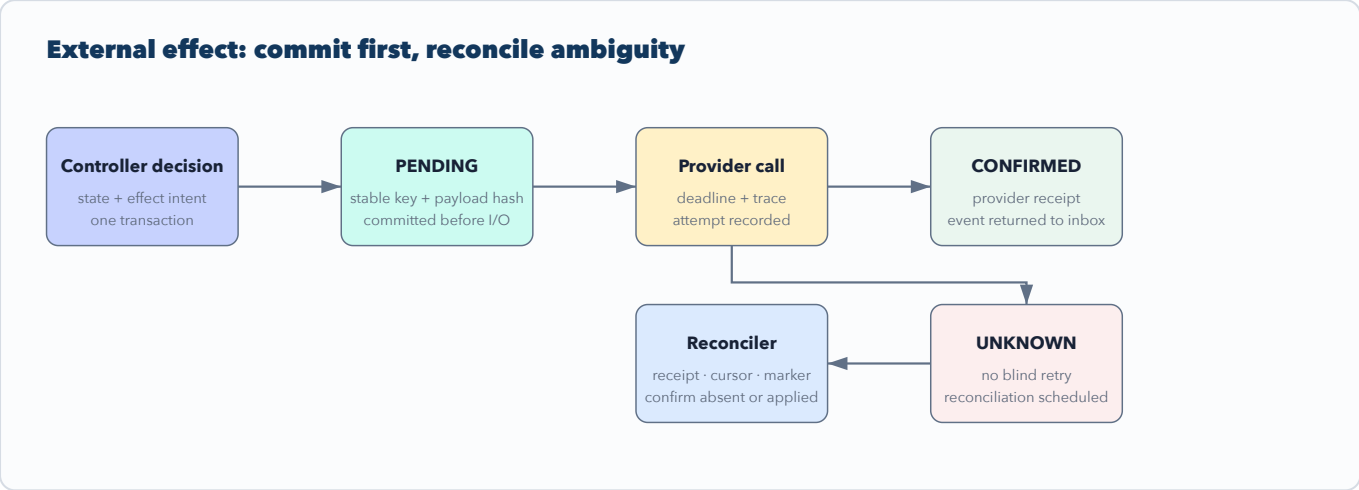
Model routing receipt

Persist task class, requested tier, selected model, fallback, reason, router version, decision latency, risk ceiling, and reviewer requirements. Model fallback may continue work only inside the new model's ceiling and never weakens proof.

SECTION 14

Effect journal and reconciliation

Phase 0 established the semantics. v2 must put story transitions and effect intents in one transaction and implement provider-specific reconciliation for every external write.



Effect state machine

`pending → confirmed`, `pending → unknown`, `pending → failed`, `unknown → confirmed`, `unknown → failed`, or `unknown → manual_review`. A confirmed or failed effect is immutable except for append-only annotations.

Provider reconciliation contracts

Provider/effect	Stable idempotency marker	Reconciliation proof
Slack message	story/effect marker in metadata or canonical body	channel/thread history by marker; exact account and destination
GitHub issue/comment/label	hidden marker + repository/object identity	API list/get, delivery ID, node ID, body hash, label state
Workboard mutation	card + operation + revision key	card/event cursor and expected revision/state
Gateway session start/send/abort	assignment run/generation idempotency key	session/run status and admitted run identity
Deployment/restart	package hash + operation + rollout generation	guarded receipt, process identity, loaded version/schema, health checks
Repository merge/close	repository + PR/issue + source SHA + action	provider-authoritative merged/closed state and resulting SHA

Reconciliation policy

- Attempt cadence is provider-specific, bounded, jittered, and visible.
- A second send occurs only after provider-authoritative proof of absence and within retry budget.
- Payload hash collision under one idempotency key fails closed and emits a security/audit event.
- Unknown effects block dependent completion when the effect is semantically required.
- Operator override requires exact evidence and cannot simply label an effect “confirmed.”

SECTION 15

Evidence ledger and proof profiles

Evidence is append-only, content-addressed, and bound to the exact producing and validating runs. Story status text, model confidence, or a Slack “done” message is never proof.

EvidenceEventV2

```
{
  evidence_id: "evd_...",
  evidence_type: "artifact | source_ref | ci | qa | provider | model_route | approval | closeout",
  story_id: "...",
  graph_version: 3,
  node_id: "...",
  assignment_id: "...",
  producing_run_id: "...",
  producing_generation: 2,
  validator_run_id: "...",
  proof_profile: "implementation_v2",
  content_hash: "sha256:...",
  source_uri_or_path: "...",
  source_sha: "...",
  schema_version: 2,
  verdict: "observed | valid | invalid | superseded",
  occurred_at: "...",
  lineage: { inputs: ["evd_..."], toolchain: "...", model_route: "..." }
}
```

Required proof profiles

Profile	Producer proof	Independent validation	Completion gate
architecture_v2	ADR, contracts, threat/failure analysis, migration impact	independent architecture review for material changes	approved decision + bounded implementation nodes
implementation_v2	source SHA, diff scope, build/typecheck/tests, artifact hashes	CI and independent QA where required	all acceptance tests and no unresolved high-risk findings
data_ml_v2	dataset/corpus hash, lineage, split, code/config/model SHAs, metrics	leakage, calibration, slice, replay, semantic review	promotion policy; never only aggregate score
qa_v2	environment, exact source ref, steps, expected/actual, fixtures	independent from producing run	typed PASS/FAIL/BLOCK with evidence
no_pr_v2	reason no PR is appropriate, exact local artifact, validation	policy-specific reviewer if material	source-of-truth still receives durable closeout
deployment_v2	package hash, preflight, rollback, drain, health baseline	loaded version/schema, process receipt, Slack/runtime health	one restart, green post-rollout canary
closeout_v2	merged/accepted provider receipts, docs/migrations, remaining risks	canonical story and dependencies reconciled	no required unknown effects or unresolved proof gaps

Proof evaluator

- Profiles are versioned code/config, not prompt text.
- The evaluator emits individual requirement verdicts and stable deficiency codes.
- Superseded artifacts remain in history but cannot satisfy a newer graph/run without explicit carry-forward policy.
- External URLs are not assumed immutable; retain provider object IDs, revisions, hashes, and retrieval receipts.
- Secrets and raw provider rows are rejected from evidence payloads; store references and safe summaries.

SECTION 16

Provider adapters and Workboard intake

Workboard, GitHub, Slack, Gateway, and sessions are sources or effect targets. None is an independent orchestration truth.

Adapter contract

- **Inbound:** authenticate → normalize → hash → persist inbox → return receipt.
- **Outbound:** claim committed effect → call with deadline → classify provider response → persist result event.
- **Read/reconcile:** provider get/list by stable marker → produce typed reconciliation evidence.
- **Health:** latency/error/cursor freshness/rate-limit state without mutating stories.

Workboard card-to-story bridge

1. Read only events after the durable board cursor in bounded pages.
2. Persist a page and every normalized event before advancing the cursor.
3. Validate CardV2 contract. Missing product intent, done conditions, risk, or proof enters triage.
4. Canonicalize repository/issue family and link duplicates to one story.
5. Compile graph version and return canonical story receipt to Workboard as a journaled effect.
6. Project story status back to Workboard as a view, never as independent truth.
7. Archive completed board cards by policy so hundreds of old done cards do not remain active intake.

Slack behavior

Keep the proven low-noise canonical thread model. Public messages are projections of durable events: takeover request, owner ACK, owner result, operator reroute, and closeout/advance. Editing or deleting Slack messages cannot rewrite control-plane history. Oscar's account is the only EWT operator identity used in the authorized channel.

GitHub behavior

GitHub remains implementation source of truth for issues, PRs, checks, reviews, merges, and closeout. The plugin captures exact provider IDs/SHAs/check runs and applies effects through idempotent markers. Provider-authoritative CI and merge state wins over cached or model-reported status.

SECTION 17

Hooks, scheduling, and resource governor

Event-driven does not mean unbounded. Hooks enqueue quickly; the scheduler admits only what the Mac mini and providers can sustain.

Hook contract

Hook/event	Enqueued fact	Budget
message_received	relevant account/channel/message/thread metadata and typed intent candidate	< 50 ms; no network/model
agent_end	terminal exact session/run event and proof-evaluation wake	< 50 ms
after_tool_call	bounded tool result metadata/progress, never raw secrets	< 50 ms
session_start/end	exact session health and assignment reconciliation event	< 50 ms
subagent lifecycle	parent delegation, child identity, progress, terminal outcome	< 50 ms

Scheduler design

- Consumes `ready_events` by priority with aging and a reserved maintenance/reconciliation slice.
- Uses a transactional claim and controller fence; nominal parallel batches must actually dispatch concurrently within limits.
- Priority order starts with live safety/health, then critical path, QA unblock, canary, normal work, and maintenance—with aging to prevent starvation.
- Broad sweep is cursor-bounded safety reconciliation only and should cause fewer than 10% of transitions.
- Durable timers replace ad hoc sleeps for SLA, lease, retry, reconciliation, and escalation wakeups.

Resource governor inputs

Signal	Action
Gateway unhealthy / queue latency high	stop admission; retain claims briefly or release by policy; reconcile existing runs
CPU or event-loop p99 high	reduce worker concurrency and batch size; protect hook latency
RSS warning/critical	stop new heavy work; cancel/checkpoint cancellable training; retain control-plane writes
Disk low or WAL growth abnormal	stop admission; checkpoint safely; verify backup; never delete evidence ad hoc
Provider errors/rate limits	provider-specific circuit breaker; schedule reconciliation; do not block unrelated providers
Shutdown	stop admission, abort cancellable I/O, flush committed receipts, return in <10 seconds

SECTION 18

Tool-surface reduction and compatibility

The 144-tool surface should become a generated compatibility layer over the 8–12-tool v2 kernel. Tool removal is a migration, not a flag-day break.

One authoritative registry

Expand the Phase 0 `ToolSpec` into a full registry containing tool name, Gateway method, public/legacy status, schema, description, command/query mapping, authority scopes, risk class, timeout, deprecation version, replacement, telemetry key, and effect category. Generate:

- runtime registrations and Gateway methods;
- `openclaw.plugin.json` activation/contracts/tool metadata;
- JSON Schema or TypeBox request/response schemas;
- documentation tables and compatibility map;
- parity/digest fixtures and changelog entries.

Compatibility policy

1. Classify every existing tool as query, command, planner, validator, adapter, composite, alias, or obsolete.
2. Map each retained behavior to a kernel command/query. Freeze legacy request and response fixtures.
3. Return deprecation metadata and usage telemetry without adding public Slack noise.
4. Keep compatibility aliases for at least two releases after v2 public tools are available.
5. Disable an alias only when usage is zero for the agreed window and its migration receipt is published.
6. Remove internal registered-tool composition first; adapters call code directly through typed interfaces.

Extraction sequence from `src/tools.ts`

Slice	New module family	Do not change simultaneously
1	`kernel/commands`, `kernel/queries`, command envelope/receipts	storage authority
2	`policies/` proof, risk, readiness, closeout, routing	public schemas
3	`controllers/story-family`, projectors, timers	legacy behavior removal
4	`adapters/` Slack/GitHub/Workboard/Gateway/deployment	provider semantics without contract tests
5	`compat/legacy-tools` generated wrappers	removing aliases before telemetry

SECTION 19

Observability and Oscar's operator experience

Oscar needs one compact operational truth: what is active, why it is or is not moving, what proof is missing, what effect is ambiguous, who owns the blocker, and when the next deterministic trigger occurs.

Operator snapshot

Control frame

- one hot lane
- prepared next lane
- QA-ready path
- ML unblocker
- blocker owner/fallback
- next review trigger

Health frame

- inbox age and cursor lag
- controller lease/fence
- ready and running WIP
- unknown effects
- provider/gateway health
- database/WAL/backup state

Why-trace

For any story/node, provide a deterministic chain:

```
card revision → canonicalization receipt → graph version/hash
→ dependency verdicts → proof requirement verdicts
→ risk/approval decision → candidate owner exclusion reasons
→ resource/WIP decision → claim/run/generation
→ effect status/receipt → result/evidence verdict
→ next ready event or blocker/escalation
```

Metrics

Category	Required measurements
Flow	queue age, phase latency, result-to-ready, result-to-dispatch, ACK/result SLA, gate closures
Correctness	duplicate suppression, stale rejections, effect unknown age, reconciliation outcomes, recovery repairs
Control	lease contention, generation takeovers, WIP denials, conflict claims, human approvals
Resources	CPU, RSS, event-loop p99, disk/WAL, gateway latency, provider errors/rate limit
Quality	proof deficiencies, QA return causes, retry causes, downgraded model routes, closeout completeness

Use bounded-cardinality identifiers. Trace IDs may be sampled in metrics but retained in durable audit. Alerts should fire on SLO breach or correctness risk, not on routine state churn.

SECTION 20

Queue repair, archive, and retention

Issue #55 is a required first repair because stale historical dependency keys can block valid bookkeeping. The database migration is the opportunity to make dependency identity and archival references explicit.

#55 repair requirements

- Reproduce the stale dependency-key mismatch from a frozen fixture without modifying live state.
- Define one canonical dependency identity: story/family/node identity, never incidental active/archive record location.
- Provide a deterministic mapping for legacy handoff IDs, issue-family keys, superseded siblings, and archived targets.
- Repair copy → verify → reference update → tombstone/delete ordering under interruption.
- Fail closed when active and archived copies diverge; never overwrite one with the other automatically.
- Prove idempotent rerun and second-pass zero-change convergence.

Retention classes

Class	Retention rule
Active control state	Never archived while referenced by nonterminal story, effect, lease, evidence, or migration.
Immutable events/evidence/audit	Retained by policy; compact only through signed export and content-hash manifest.
Provider payloads	Store safe normalized data and hashes; omit secrets/raw rows; retention honors provider/privacy boundary.
Completed projections	May be rebuilt from events; archive for query speed after reference checks.
Unknown effects/dead letters	Never age out silently; require confirmed resolution or explicit operator disposition.
Legacy JSON	Read-only migration evidence until explicit cleanup approval after v2 soak.

Workboard hygiene

Separate intake cards, canonical story projection, and archive views. Completed cards should be archived by cursor-driven policy after closeout proof; old done cards must not remain in active queue scoring or force the new schema to mimic historical noise.

SECTION 21

Security and human approval boundary

v2 may increase autonomy only inside explicit, server-enforced authority. Prompt text can never expand it.

Action	Autonomous when	Human required when
Read/analyze	assigned repository/data scope and privacy class permit it	new sensitive/external data or access boundary
Edit/test	leased worktree/path, bounded node, no unowned dirty-state overwrite	ownership conflict cannot be safely isolated
Merge/close	low/medium risk, independent proof complete, provider CI authoritative	high risk, material architecture, policy exception, downgraded strong review
Deploy/restart	fixed operation, package verified, rollback ready, drain/health green	new exposure, irreversible state, unsafe health, second restart, production exception
External write	approved destination/template, journaled effect, privacy class permitted	new audience, public publication, legal/privacy ambiguity
Spend/credentials/access	never implicit	always explicit new authority

Threats v2 must test

- Caller claims to be Oscar, chooses actor/owner, or injects webhook identity.
- Path traversal, symlink escape, alternate repository root, or command-class smuggling.
- Delegation replay, expired token, revoked manifest, wrong run/generation, excess depth.
- Payload/idempotency collision and forged reconciliation receipt.
- Cross-story evidence attachment and QA self-approval.
- SQL injection, unsafe dynamic identifiers, migration checksum tampering, database replacement.
- Secret/raw provider data entering logs, evidence, metrics, dead letters, or Slack.
- Resource-exhaustion card: unbounded DAG, fanout, retries, output size, or training job.

> The deployment supervisor must continue using the mandatory safe gateway restart gate. Never call a forced gateway restart during plugin rollout. Build, typecheck, test, pack, inspect, install once, drain, restart once, and verify Slack/runtime health. A perceived need for a second restart is a diagnosis blocker.

SECTION 22

Recovery and chaos verification

Autonomy is safe only when interruption at every boundary converges without duplicate work or lost evidence.

Mandatory interruption points

Boundary	Injected failure	Expected recovery
Inbox	crash after event commit, before claim	event claimed once after restart
Controller	crash mid-transaction	atomic rollback; event remains processable
Ready event	crash after node completion commit	ready event dispatches once
Claim	two schedulers race	one assignment/run; loser returns original/current receipt
Session dispatch	provider accepts, response lost	unknown; exact run reconciled; no blind relaunch
Worker	deadline, crash, late success	cancel/retry by policy; stale result rejected
Slack/GitHub/Workboard	write accepted, timeout	provider marker confirms applied; no duplicate
Database	busy, disk full, WAL growth, corruption	fail closed, health alert, verified restore path
Lease	controller pauses past expiry then resumes	old fence cannot mutate
Graph	supersession during running node	generation advances; uncommitted results become harmless
Shutdown	provider hangs indefinitely	abort signal propagates; process returns in <10 seconds
Migration	crash after partial import/cutover step	checksum/idempotency resumes or rolls back deterministically

Recovery pass contract

1. Verify schema/migrations and database integrity.
2. Expire or reconcile stale leases and claims using fences.
3. Resume unprocessed inbox events.
4. Reconcile pending/unknown effects by provider policy.
5. Rebuild/verify ready-event and projection consistency.
6. Reconcile sessions/runs and resource claims.
7. Emit a recovery receipt with every repair.
8. Immediately run a second dry reconciliation pass; it must report zero repairs and zero new effects.

SECTION 23

EWT first vertical-slice contract

EWT is the proving customer, not part of the kernel. Its domain packages should plug into generic card, DAG, evidence, policy, and adapter interfaces.

Correct product boundary

> Deterministic Elliott and price rules alone may hard-invalidate a wave alternative. ML estimates survival/invalidation risk, marks a still-valid candidate `at_risk`, and reranks surviving alternatives. It never invents count truth or reverses a deterministic breach.

Required EWT plugin integration packages

Package	Handoff responsibility	Domain authority
EWT Card templates	Typed templates for architecture, implementation, data/ML, QA, deployment, closeout.	Oscar/product intent and risk.
EWT DAG compiler extension	Dependency graph for lifecycle schema, corpus, evaluator, model, API/UI, QA, soak.	Compiler policy; no direct execution.
EWT proof profiles	Candidate identity, candle lineage, invalidation/risk events, replay, CI/QA/promotion packet.	Deterministic runtime and independent QA.
EWT capability labels	Architecture, ML semantics, implementation, domain QA, data lineage, platform.	Authoritative agent registry.
EWT projectors	Hot lane, corpus gate, training gate, QA path, product closeout.	Views only.
EWT provider adapters	GitHub issue/PR/check and approved Workboard/Slack projections.	Provider receipts.

First canary story

Choose a low-risk, bounded repository change with deterministic tests and independent QA—large enough to exercise intake, graph, implementation, proof, result, next-node wake, GitHub/Workboard/Slack effects, closeout, restart, and replay; small enough that rollback is trivial. Do not use live trading, model promotion, credentials, production exposure, or large training as the first canary.

Later EWT product nodes

- CandidateIdentityV1 and immutable per-candle lifecycle event schemas.
- Reviewed alternate/correction corpus gate before invalidation-model training.
- Historical/incremental replay equivalence and exactly one first-breach hard invalidation event.
- Temporal and family-disjoint evaluation, calibration, ambiguity/censoring policy, drift/explainability.
- API/UI explicit states: active, at_risk, hard_invalidated, target_hit, same_bar_ambiguous, censored/no_signal.
- Independent QA, soak, rollback, and bounded promotion packet.

These later nodes validate the v2 control plane but are not required to declare the generic plugin complete.

SECTION 24

Delivery roadmap and phase gates

Each phase must land as a reviewable vertical slice with a rollback point. Do not accumulate an unreviewable rewrite.

Phase 1 — Baseline, #55 repair, and kernel seam

Build: exact 0.1.100 baseline, frozen fixtures, issue #55 repair, command/query envelope, repository interfaces, architecture dependency tests, generated ToolSpec metadata extension.

Exit: all 0.1.100 tests plus new #55 and kernel-contract tests; no behavior change in live adapters.

Phase 2 — SQLite journal and parity migration

Build: migrations, core tables/indexes, command receipts, event/evidence/effect repositories, backup/restore, importer, parity queries, shadow mode.

Exit: 100% classified parity for required projections; integrity/restore/restart convergence green; JSON still authoritative.

Phase 3 — Durable inbox and singular family controller

Build: inbox, cursors, event normalizers, family lease/fence, controller transaction, reason codes, ready-event outbox, durable timers.

Exit: races, stale lease, crash boundaries, duplicate events, and second-pass convergence green in isolated canaries.

Phase 4 — Card/DAG, capabilities, and bounded dispatcher

Build: CardV2, deterministic compiler, versioned graph, lazy materialization, unified identity manifests, delegation, WIP/conflict/resource governor, model-route receipts.

Exit: one bounded story reaches independent QA without broad sweep; forbidden owner/path/risk routes fail closed.

Phase 5 — Provider reconciliation and small public surface

Build: provider-specific reconcilers, 8–12 v2 tools, generated compatibility wrappers, telemetry/deprecation, Workboard cursor intake, low-noise projections.

Exit: ambiguous-effect matrix green; legacy contract fixtures pass; no registered-tool recursion; surface parity generated from registry.

Phase 6 — Event hooks, operator control, and production canary

Build: bounded hooks, scheduler fairness, health/circuit breakers, operator snapshot/why-trace, metrics/SLOs, EWT low-risk vertical slice, soak and chaos replay.

Exit: final Definition of Done, one guarded production rollout/restart, live version/schema/parity, Slack/task health, no duplicate effects, and signed closeout packet.

Approval boundaries

Codex may implement and test in isolation. Human approval remains required for live installation/restart, new services or cost, credentials/access, privacy-sensitive external data, irreversible cleanup, production exposure, and material product/architecture exceptions.

SECTION 25

Complete definition of done

Do not label v2 complete until every applicable box is proven by a durable artifact.

Architecture and state

- ☐ one mutation kernel and family controller
- ☐ SQLite/WAL authoritative
- ☐ all migrations checksum-verified
- ☐ backup and restore proven
- ☐ JSON is read-only evidence
- ☐ #55 and legacy dependency fixtures green
- ☐ second recovery pass is zero-change

Events and effects

- ☐ every inbound source persists before act
- ☐ provider cursor advancement is durable
- ☐ all external writes use effect journal
- ☐ unknown outcomes retain reconciliation
- ☐ no blind duplicate provider write
- ☐ effect/key collision fails closed
- ☐ pending/unknown effects visible to Oscar

Stories and workers

- ☐ CardV2 canonicalizes idempotently
- ☐ DAG is deterministic and versioned
- ☐ claims include exact run/generation/fence
- ☐ WIP/conflicts/resources are enforced
- ☐ delegation is exact, expiring, replay-safe
- ☐ stale results cannot mutate
- ☐ cancellation reaches provider/tool/model seams

Evidence and policy

- ☐ evidence is append-only and content-addressed
- ☐ proof profiles are versioned
- ☐ CI/QA/provider authority is exact
- ☐ QA independence is enforced
- ☐ models cannot decide hard gates
- ☐ fallback cannot weaken proof
- ☐ high-risk completion remains human-gated

Surface and operations

- ☐ 8–12 public v2 tools
- ☐ legacy adapters generated and measured
- ☐ no registered-tool recursion
- ☐ manifest/runtime/Gateway/docs generated
- ☐ hooks enqueue under 50 ms p99
- ☐ shutdown returns under 10 seconds
- ☐ broad sweep causes under 10% of transitions

Release proof

- ☐ full local suite green
- ☐ hosted CI green
- ☐ independent security/architecture review SHIP
- ☐ migration and rollback rehearsal green
- ☐ EWT low-risk canary closes end-to-end
- ☐ soak meets latency/correctness SLOs
- ☐ one guarded restart and live health green

> "Tests green" is not sufficient if tests encode unsafe behavior. Review must independently challenge path scope, authority, ambiguity, reconciliation proof, stale fencing, destructive migration, and provider truth—exactly as the Phase 0 review did.

SECTION 26

Implementation backlog

Create one epic and these bounded child issues. Each child issue should include exact source ref, files/seams, tests, proof artifact, rollback, and exit gate.

ID	Priority	Work package	Primary output
V2-01	P0	Resolve exact 0.1.100 source baseline and freeze proof	baseline receipt and clean v2 worktree
V2-02	P0	Repair #55 dependency identity/closeout defect	migration-safe canonical dependency mapping
V2-03	P0	Introduce command/query envelope and repository boundaries	single mutation seam with architecture tests
V2-04	P0	Create SQLite connection, pragmas, migrations, health	versioned WAL database
V2-05	P0	Implement core story/graph/node/assignment/lease schema	transactional domain repositories
V2-06	P0	Implement inbox/effect/evidence/audit/dead-letter journals	append-only journals and indexes
V2-07	P0	Build JSON inventory/import/checkpoint tooling	signed migration manifest and importer
V2-08	P0	Shadow-read and normalized parity harness	classified parity report
V2-09	P0	Backup, restore, disk-full, integrity and rollback proof	recovery packet
V2-10	P1	Provider/native event envelope and cursor inbox	deduplicated durable intake
V2-11	P1	Family controller lease and fencing	single-owner saga controller
V2-12	P1	Controller transaction, domain events, reason codes	deterministic transition engine
V2-13	P1	Ready-event outbox and durable timers	event-driven scheduler feed
V2-14	P1	CardV2 schemas and canonicalization	idempotent card-to-story receipt
V2-15	P1	Versioned DAG compiler and graph migration	deterministic graph hash and lazy nodes
V2-16	P1	Agent identity/capability registry	one authoritative owner manifest
V2-17	P1	Delegation capability issue/validate/revoke	bounded assignment authority
V2-18	P1	WIP/conflict/resource admission governor	safe dispatcher admission
V2-19	P1	Gateway/session exact dispatch reconciler	no duplicate unknown launch
V2-20	P1	Slack/GitHub/Workboard effect reconcilers	provider-authoritative unknown resolution
V2-21	P1	EvidenceEventV2 and proof evaluator	exact immutable proof ledger
V2-22	P2	Generated v2 tool registry and 8–12 public tools	small stable external surface
V2-23	P2	Legacy compatibility generation and telemetry	two-release migration bridge
V2-24	P2	Workboard cursor card compiler bridge	complete card-to-story intake
V2-25	P2	Native hook enqueue adapters	sub-50 ms event capture
V2-26	P2	Fair scheduler, circuits, resource metrics	bounded event-driven dispatch
V2-27	P2	Operator snapshot, why-trace, SLOs	Oscar control surface
V2-28	P2	Chaos/restart/migration fault matrix	second-pass convergence proof
V2-29	P2	EWT low-risk vertical canary package	end-to-end product proof
V2-30	P0	Independent ship review and guarded rollout	v2 release, live verification, closeout packet

SECTION 27

Codex 5.6 Sol Ultra execution prompt

Paste the text below into the implementation session and attach this PDF.

```
You are implementing OpenClaw Handoff Control Plane v2 for Phil.

Treat the attached “OpenClaw Handoff Control Plane v2 — Complete Implementation
Specification” as the normative product, architecture, migration, testing, and
release contract. Read it completely before editing.

Critical baseline:
- The live shipped baseline is package 0.1.100 / schema 2026-07-25.1 from PR #54.
- Do not build from the older dirty 0.1.99 local checkout or a stale origin/main
  that may report 0.1.96.
- First locate and verify the exact merged/release commit containing Phase 0:
  ExecutionContext deadlines/cancellation, server-derived scoped authority,
  fixed deployment operations, durable effect intents and reconciliation,
  unknown-dispatch fail-closed behavior, and ToolSpec parity.
- Preserve every dirty checkout, worktree, branch, untracked file, installed
  runtime, and state directory. Create an isolated v2 worktree.

Delivery model:
- Create one GitHub epic and bounded child issues matching V2-01 through V2-30.
- Work in ordered phases. Keep each change reviewable and rollback-safe.
- Do not perform a greenfield rewrite. Extract the typed kernel behind
  compatibility adapters while preserving proven invariants.
- One family controller and one mutation path are mandatory. Hooks, cron,
  Workboard, Slack, GitHub, canaries, and sweeps only enqueue facts.
- SQLite/WAL becomes authoritative only after inventory, #55 repair, import,
  parity, shadow mode, backup/restore, and scoped canary gates.
- Do not create indefinite dual-write or a second active controller.
- No arbitrary shell, caller-selected authority, blind unknown-effect retry,
  model-decided completion, or evidence based on prose.

Quality:
- For every work package: identify invariant, durable records, idempotency key,
  authority, deadline/cancellation, failure states, reconciliation/retry,
  restart behavior, observability, tests, rollback, and exact exit evidence.
- Add race, crash, stale-generation, forged-authority, payload-collision,
  provider-ambiguity, disk/database, migration interruption, and second-pass
  convergence tests.
- Tests must not encode unsafe behavior. Request an independent adversarial
  review before rollout and fix every blocker, even if the suite is green.
- Keep exact proof in GitHub: source SHA, CI, artifacts, migration reports,
  canary receipts, and remaining risks.

Live safety:
- Implementation and isolated canaries are authorized only within the existing
  project boundaries. Escalate new cost/services, credentials/access,
  privacy-sensitive data, production/irreversible risk, and material product or
  architecture exceptions.
- Do not install or restart the live plugin until every final gate passes.
- Build, typecheck, test, pack, and inspect first. Install once. Use the mandatory
  safe gateway restart procedure. One rollout gets at most one restart.
- After restart verify live version/schema, strict catalog/runtime parity,
  database/migration/backup health, event/controller/effect health, task state,
  and Oscar Slack connectivity.

Completion:
- Do not call v2 complete until every checkbox in Section 25 is backed by a
  durable artifact and the low-risk EWT vertical canary closes end to end.
- EWT model promotion is not required for plugin v2 completion. Deterministic
  Elliott invalidation remains authoritative; ML may only mark at-risk and
  rerank still-valid alternatives.

Begin with a read-only preflight and report the exact baseline, dirty-state
preservation plan, proposed issue tree, phase order, and first proof gate before
making source changes. Then proceed autonomously through the authorized work,
stopping only for a true human-owned boundary.
```

APPENDIX A

Proposed schemas and indexes

Names may change, but equivalent uniqueness, foreign-key, append-only, and fencing behavior is mandatory.

```
PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;
PRAGMA busy_timeout = 5000;

CREATE TABLE stories (
  story_id TEXT PRIMARY KEY,
  canonical_family_key TEXT NOT NULL UNIQUE,
  status TEXT NOT NULL,
  risk TEXT NOT NULL,
  priority INTEGER NOT NULL,
  version INTEGER NOT NULL,
  controller_generation INTEGER NOT NULL,
  current_graph_version INTEGER,
  intent_json TEXT NOT NULL,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE event_inbox (
  inbox_id INTEGER PRIMARY KEY AUTOINCREMENT,
  source TEXT NOT NULL,
  event_id TEXT NOT NULL,
  revision TEXT NOT NULL DEFAULT '',
  event_type TEXT NOT NULL,
  schema_version INTEGER NOT NULL,
  payload_hash TEXT NOT NULL,
  payload_json TEXT NOT NULL,
  signature_status TEXT NOT NULL,
  principal_id TEXT,
  source_cursor TEXT,
  trace_id TEXT NOT NULL,
  occurred_at TEXT,
  received_at TEXT NOT NULL,
  state TEXT NOT NULL,
  claim_generation INTEGER NOT NULL DEFAULT 0,
  processed_at TEXT,
  UNIQUE(source, event_id, revision)
);

CREATE TABLE leases (
  scope_key TEXT PRIMARY KEY,
  owner_id TEXT NOT NULL,
  generation INTEGER NOT NULL,
  acquired_at TEXT NOT NULL,
  renewed_at TEXT NOT NULL,
  expires_at TEXT NOT NULL,
  metadata_json TEXT NOT NULL
);

CREATE TABLE effect_intents (
  idempotency_key TEXT PRIMARY KEY,
  story_id TEXT REFERENCES stories(story_id),
  effect_type TEXT NOT NULL,
  provider TEXT NOT NULL,
  operation TEXT NOT NULL,
  payload_hash TEXT NOT NULL,
  payload_json TEXT NOT NULL,
  status TEXT NOT NULL,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  confirmed_at TEXT,
  provider_receipt_json TEXT,
  reconciliation_policy TEXT NOT NULL,
  next_reconcile_at TEXT,
  UNIQUE(provider, operation, idempotency_key)
);

CREATE INDEX idx_inbox_ready
  ON event_inbox(state, received_at, inbox_id);
CREATE INDEX idx_effect_unknown
  ON effect_intents(status, next_reconcile_at);
CREATE INDEX idx_story_status_priority
```

```
  ON stories(status, priority, updated_at);
CREATE UNIQUE INDEX idx_one_active_assignment_per_node_generation
  ON assignments(story_id, graph_version, node_id, generation)
  WHERE state IN ('claimed','dispatch_pending','dispatch_unknown','running');
```

Additional constraints

- Use CHECK constraints for enums and nonnegative versions/attempts where supported.
- Evidence and audit tables should be append-only through repository APIs; triggers may provide defense in depth.
- Canonical JSON hashing must define key order and normalization. Do not hash ordinary `JSON.stringify` if producers may vary key order.
- Large artifacts stay in repositories/object storage; database stores identity, hash, safe metadata, and retrieval proof.
- Use migration checksums and refuse to run if an applied migration's content changes.

APPENDIX B

Acceptance-test matrix

Area	Minimum required tests
Canonical identity	duplicate cards, renamed titles, multiple provider sources, issue-family collision, supersession, archived legacy IDs
Database	migration order/checksum, WAL restart, busy timeout, disk full, corrupted copy, foreign keys, backup/restore, old schema refusal
Commands	idempotent retry, key/payload collision, expected-version conflict, unauthorized principal, expired capability, audit receipt
Inbox	duplicate delivery, reordered revision, invalid signature, cursor page interruption, poison event, schema upgrade, claim takeover
Controller	lease race, stale fence, crash mid-transaction, no-op reason, deterministic replay, controller takeover, second-pass convergence
DAG	deterministic hash, cycle, missing output, unbounded fanout, join policies, lazy materialization, graph migration, supersession
Dispatch	WIP limit, conflict group, resource denial, unhealthy owner, unknown session send, exact-run reconcile, late result, cancellation
Delegation	actor forgery, token replay, nonce reuse, expiry, revocation, excess depth, path escape, risk/model ceiling, wrong run/generation
Effects	accept-then-timeout for every provider, confirmed duplicate, authoritative absence then retry, forged receipt, collision, manual review
Evidence	cross-story attachment, cross-generation proof, mutated artifact, missing SHA, self-QA, superseded carry-forward, secret/raw-row rejection
Compatibility	all legacy request fixtures, response normalization, deprecation metadata, no recursive registered-tool call, generated surface parity
Hooks	latency under provider outage, no network/model path, secret filtering, duplicate hook event, queue-unavailable health failure
Resources	CPU/RSS/disk/gateway/provider circuits, fairness aging, maintenance slice, training singleton, shutdown under hung provider
Rollout	package hash, installed/runtime version, migration preflight, rollback rehearsal, one restart, Slack/runtime health, live canary

Required proof levels

- **Unit:** pure policy, schema, hashing, compiler, reason codes.
- **Transactional integration:** real SQLite temp database, concurrent connections, crash/reopen, migration fixtures.
- **Adapter contract:** recorded provider responses and ambiguity cases; no live writes required.
- **End-to-end isolated:** card → graph → dispatch stub → result → proof → next node → close.
- **Chaos:** deterministic failpoints at every commit/I/O boundary.
- **Production canary:** bounded low-risk EWT story with real provider receipts and guarded rollout proof.

APPENDIX C

Explicit non-goals

The following should not enter the v2 critical path unless a separate approved decision changes the boundary.

- Replacing OpenClaw's session runtime, Workboard, GitHub, Slack, or the model router.
- Building a generic distributed workflow platform for unrelated organizations or cloud-scale multi-region operation.
- True exactly-once external delivery. The honest contract is at-least-once events plus idempotent mutation, duplicate suppression, provider reconciliation, and exactly-once logical transition.
- Cryptographic infrastructure beyond the trust boundary's needs. Tamper-evident database-backed capabilities are sufficient unless tokens cross machines/services in a way that requires public-key verification.
- Indefinite dual-write between JSON and SQLite.
- Migrating every completed Workboard card or historical backup into active transactional truth.
- Removing all 144 legacy tools in the first v2 release.
- Increasing worker concurrency before resource and correctness soak earns it.
- Automatic credentials, permission expansion, provider spending, public publication, or privacy-sensitive ingestion.
- Automatic high-risk/material architecture merge or production rollout without the defined human gate.
- Making ML responsible for Elliott Wave hard invalidation or claiming model promotion from aggregate metrics alone.
- Cleaning existing EWT/plugin worktrees or uncommitted files as part of migration.

> **Final directive to the implementation team:** finish the smallest durable control plane that satisfies every invariant and proof gate. Speed should come from singular authority, transactionality, and event-driven wakeups—not from more background initiators, more tools, more prompt conventions, or more concurrency.

Evidence basis: the 25 July 2026 planning report, live rollout record for 0.1.100, local read-only inspection of the Phase 0 source branch, current module/tool counts, existing completion plan, and EWT operating boundaries. No source, runtime, GitHub, Workboard, Slack, plugin, or configuration changes were made to prepare this document.